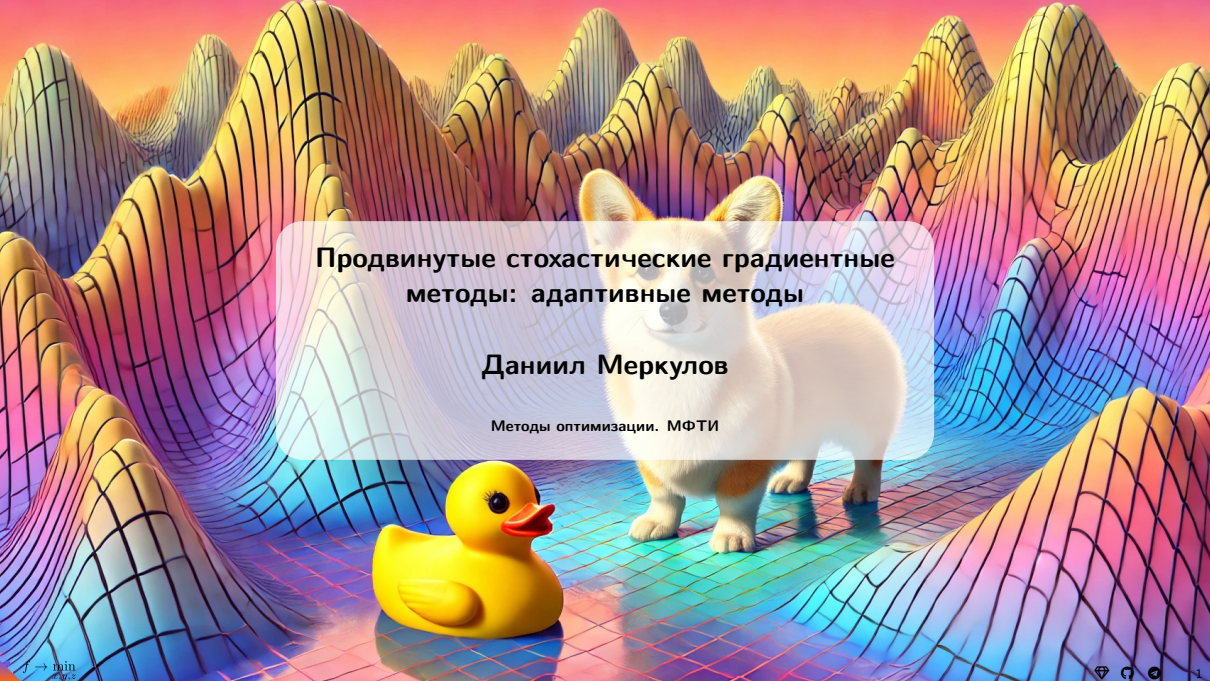


The background of the slide is a vibrant, stylized illustration. It features a series of colorful, wavy hills or mountains in shades of yellow, orange, pink, and blue. The surface of these hills is covered in a grid-like pattern of black lines. In the foreground, a small, fluffy Corgi dog with white fur and orange patches is standing on a similar grid-like surface. To the left of the dog, there is a small, bright yellow rubber duck. The overall scene is set against a soft, pastel-colored sky.

Продвинутые стохастические градиентные методы: адаптивные методы

Даниил Меркулов

Методы оптимизации. МФТИ

Вспоминаем задачу минимизации конечной суммы

Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .

Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .
- Сходимость при постоянном α или линейном поиске.

Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .
- Сходимость при постоянном α или линейном поиске.

Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по n .
- Сходимость при постоянном α или линейном поиске.

Перейдём от полного вычисления градиента к его несмещённой оценке, случайно выбирая индекс i_k на каждой итерации равновероятно:

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x_k) \quad (\text{SGD})$$

При $p(i_k = i) = \frac{1}{n}$ стохастический градиент является несмещённой оценкой градиента:

$$\mathbb{E}[\nabla f_{i_k}(x)] = \sum_{i=1}^n p(i_k = i) \nabla f_i(x) = \sum_{i=1}^n \frac{1}{n} \nabla f_i(x) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(x) = \nabla f(x)$$

Это означает, что математическое ожидание стохастического градиента равно истинному градиенту $f(x)$.

Адаптивные методы

Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, а обновление для $j = 1, \dots, p$ задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует тонкой настройки шага: $\alpha > 0$ задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.

Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, а обновление для $j = 1, \dots, p$ задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует тонкой настройки шага: $\alpha > 0$ задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.

Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, а обновление для $j = 1, \dots, p$ задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует тонкой настройки шага: $\alpha > 0$ задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.
- Может радикально превосходить SGD на разреженных задачах.

Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, а обновление для $j = 1, \dots, p$ задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует тонкой настройки шага: $\alpha > 0$ задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.
- Может радикально превосходить SGD на разреженных задачах.
- Главный недостаток — монотонное накопление градиентов в знаменателе. AdaDelta, Adam, AMSGrad и другие методы пытаются это исправить; они особенно популярны при обучении глубоких нейронных сетей.

Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, а обновление для $j = 1, \dots, p$ задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- AdaGrad не требует тонкой настройки шага: $\alpha > 0$ задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.
- Может радикально превосходить SGD на разреженных задачах.
- Главный недостаток — монотонное накопление градиентов в знаменателе. AdaDelta, Adam, AMSGrad и другие методы пытаются это исправить; они особенно популярны при обучении глубоких нейронных сетей.
- Константу ϵ обычно берут равной 10^{-6} , чтобы избежать деления на ноль и слишком больших шагов.

RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, которое борется с его слишком агрессивным монотонным уменьшением шага. Использует скользящее среднее квадратов градиентов, чтобы адаптировать шаг для каждого параметра. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, тогда обновление для $j = 1, \dots, p$ имеет вид:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- RMSPProp делит шаг для каждого параметра на скользящее среднее величин недавних градиентов этого параметра.

RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, которое борется с его слишком агрессивным монотонным уменьшением шага. Использует скользящее среднее квадратов градиентов, чтобы адаптировать шаг для каждого параметра. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, тогда обновление для $j = 1, \dots, p$ имеет вид:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- RMSPProp делит шаг для каждого параметра на скользящее среднее величин недавних градиентов этого параметра.
- Даёт более гибкую адаптацию шагов, чем AdaGrad, поэтому хорошо подходит для нестационарных задач.

RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, которое борется с его слишком агрессивным монотонным уменьшением шага. Использует скользящее среднее квадратов градиентов, чтобы адаптировать шаг для каждого параметра. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, тогда обновление для $j = 1, \dots, p$ имеет вид:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Замечания:

- RMSPProp делит шаг для каждого параметра на скользящее среднее величин недавних градиентов этого параметра.
- Даёт более гибкую адаптацию шагов, чем AdaGrad, поэтому хорошо подходит для нестационарных задач.
- Часто используется при обучении нейронных сетей, особенно рекуррентных.

Adadelta (Zeiler, 2012)

Расширение RMSProp, которое пытается уменьшить зависимость от вручную заданного глобального шага. Вместо накопления всех прошлых квадратов градиентов Adadelta ограничивает окно накопления некоторым фиксированным размером w . Механизм обновления не требует явного задания шага α :

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$\tilde{g}_j^{(k)} = \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)}$$

$$x_j^{(k)} = x_j^{(k-1)} - \tilde{g}_j^{(k)}$$

$$\Delta x_j^{(k)} = \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2$$

Замечания:

- Adadelta адаптирует шаги, опираясь на скользящее окно обновлений градиента, а не на сумму всех прошлых градиентов. За счёт этого шаги устойчивее к изменениям динамики модели.

Adadelta (Zeiler, 2012)

Расширение RMSProp, которое пытается уменьшить зависимость от вручную заданного глобального шага. Вместо накопления всех прошлых квадратов градиентов Adadelta ограничивает окно накопления некоторым фиксированным размером w . Механизм обновления не требует явного задания шага α :

$$\begin{aligned}v_j^{(k)} &= \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2 \\ \tilde{g}_j^{(k)} &= \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)} \\ x_j^{(k)} &= x_j^{(k-1)} - \tilde{g}_j^{(k)} \\ \Delta x_j^{(k)} &= \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2\end{aligned}$$

Замечания:

- Adadelta адаптирует шаги, опираясь на скользящее окно обновлений градиента, а не на сумму всех прошлых градиентов. За счёт этого шаги устойчивее к изменениям динамики модели.
- Метод не требует выбора начального шага, что упрощает настройку.

Adadelta (Zeiler, 2012)

Расширение RMSProp, которое пытается уменьшить зависимость от вручную заданного глобального шага. Вместо накопления всех прошлых квадратов градиентов Adadelta ограничивает окно накопления некоторым фиксированным размером w . Механизм обновления не требует явного задания шага α :

$$\begin{aligned}v_j^{(k)} &= \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2 \\ \tilde{g}_j^{(k)} &= \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)} \\ x_j^{(k)} &= x_j^{(k-1)} - \tilde{g}_j^{(k)} \\ \Delta x_j^{(k)} &= \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2\end{aligned}$$

Замечания:

- Adadelta адаптирует шаги, опираясь на скользящее окно обновлений градиента, а не на сумму всех прошлых градиентов. За счёт этого шаги устойчивее к изменениям динамики модели.
- Метод не требует выбора начального шага, что упрощает настройку.
- Часто применяется в глубоком обучении, где масштабы параметров могут сильно различаться между слоями.

Adam (Kingma and Ba, 2014) ^{1 2}

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.

Adam (Kingma and Ba, 2014) ^{1 2}

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире

Adam (Kingma and Ba, 2014) ^{1 2}

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе

Adam (Kingma and Ba, 2014) ^{1 2}

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе
- Не сходится для некоторых простых задач (даже выпуклых)

Adam (Kingma and Ba, 2014) ^{1 2}

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе
- Не сходится для некоторых простых задач (даже выпуклых)
- Почему-то очень хорошо работает для некоторых сложных задач

Adam (Kingma and Ba, 2014)^{1 2}

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

Замечания:

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе
- Не сходится для некоторых простых задач (даже выпуклых)
- Почему-то очень хорошо работает для некоторых сложных задач
- Гораздо лучше работает для языковых моделей, чем для задач компьютерного зрения: почему?

¹Adam: A Method for Stochastic Optimization

²On the Convergence of Adam and Beyond

AdamW (Loshchilov & Hutter, 2017)

Решает типичную проблему ℓ_2 -регуляризации в адаптивных оптимизаторах вроде Adam. При стандартной ℓ_2 -регуляризации к функции потерь добавляется $\lambda\|x\|^2$, что даёт в градиенте дополнительный член λx . В Adam этот член масштабируется адаптивным шагом $\left(\sqrt{\hat{v}_j} + \epsilon\right)$, из-за чего убывание весов оказывается связано с величинами градиентов.

AdamW отделяет убывание весов от шага адаптации по градиенту.

Правило обновления:

$$\begin{aligned}m_j^{(k)} &= \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)} \\v_j^{(k)} &= \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2 \\ \hat{m}_j &= \frac{m_j^{(k)}}{1 - \beta_1^k}, \quad \hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k} \\ x_j^{(k)} &= x_j^{(k-1)} - \alpha \left(\frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)\end{aligned}$$

Замечания:

- Слагаемое убывания весов $\lambda x_j^{(k-1)}$ добавляется *после* адаптивного градиентного шага.

AdamW (Loshchilov & Hutter, 2017)

Решает типичную проблему ℓ_2 -регуляризации в адаптивных оптимизаторах вроде Adam. При стандартной ℓ_2 -регуляризации к функции потерь добавляется $\lambda\|x\|^2$, что даёт в градиенте дополнительный член λx . В Adam этот член масштабируется адаптивным шагом $\left(\sqrt{\hat{v}_j} + \epsilon\right)$, из-за чего убывание весов оказывается связано с величинами градиентов.

AdamW отделяет убывание весов от шага адаптации по градиенту.

Правило обновления:

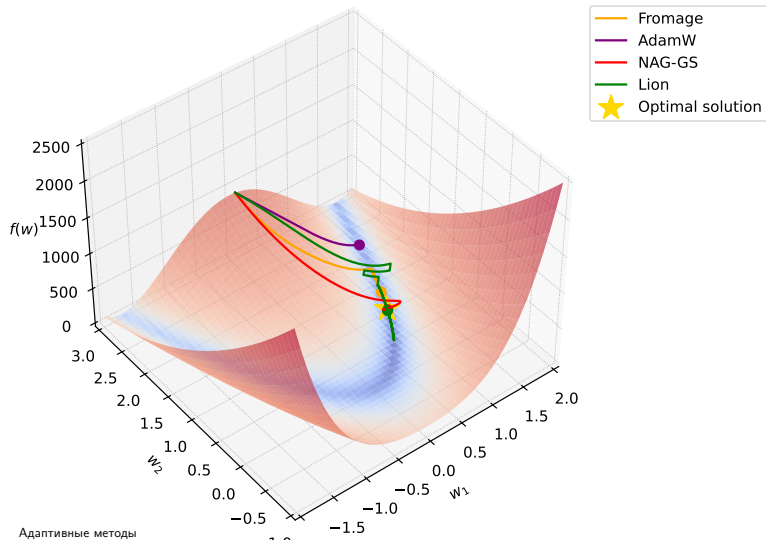
$$\begin{aligned}m_j^{(k)} &= \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)} \\v_j^{(k)} &= \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2 \\ \hat{m}_j &= \frac{m_j^{(k)}}{1 - \beta_1^k}, \quad \hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k} \\ x_j^{(k)} &= x_j^{(k-1)} - \alpha \left(\frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)\end{aligned}$$

Замечания:

- Слагаемое убывания весов $\lambda x_j^{(k-1)}$ добавляется *после* адаптивного градиентного шага.
- Широко используется при обучении трансформеров и других больших моделей. Часто является выбором по умолчанию в Hugging Face Trainer.

Таких методов немало

Rosenbrock Function.
Adaptive stochastic gradient algorithms.
Learning rate 0.003



Как их сравнивать? Бенчмарк AlgoPerf^{3 4}

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:

Как их сравнивать? Бенчмарк AlgoPerf ³ ⁴

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
 - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.

Как их сравнивать? Бенчмарк AlgoPerf ³ ⁴

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
 - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.
 - **Self-Tuning:** моделирует автоматическую настройку на одной машине (фиксированная или inner-loop настройка, бюджет в 3 раза больше). Оценка строится по медиане времени работы на 5 исследованиях.

Как их сравнивать? Бенчмарк AlgoPerf^{3 4}

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
 - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.
 - **Self-Tuning:** моделирует автоматическую настройку на одной машине (фиксированная или inner-loop настройка, бюджет в 3 раза больше). Оценка строится по медиане времени работы на 5 исследованиях.
- **Система оценивания:** результаты по задачам агрегируются с помощью профилей производительности. Профиль показывает долю задач, решённых за время не более чем в τ раз больше времени лучшей посылки. Итоговый балл — нормированная площадь под профилем (1.0 означает лучший результат на всех задачах).

Как их сравнивать? Бенчмарк AlgoPerf^{3 4}

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
 - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.
 - **Self-Tuning:** моделирует автоматическую настройку на одной машине (фиксированная или inner-loop настройка, бюджет в 3 раза больше). Оценка строится по медиане времени работы на 5 исследованиях.
- **Система оценивания:** результаты по задачам агрегируются с помощью профилей производительности. Профиль показывает долю задач, решённых за время не более чем в τ раз больше времени лучшей посылки. Итоговый балл — нормированная площадь под профилем (1.0 означает лучший результат на всех задачах).
- **Вычислительная стоимость:** для оценивания потребовалось $\sim 49,240$ суммарных часов на конфигурации из 8 NVIDIA V100 (в среднем ~ 3469 ч на одну посылку в режиме External Tuning и ~ 1847 ч на одну посылку в режиме Self-Tuning).

³Benchmarking Neural Network Training Algorithms

⁴Accelerating neural network training: An analysis of the AlgoPerf competition

Бенчмарк AlgoPerf

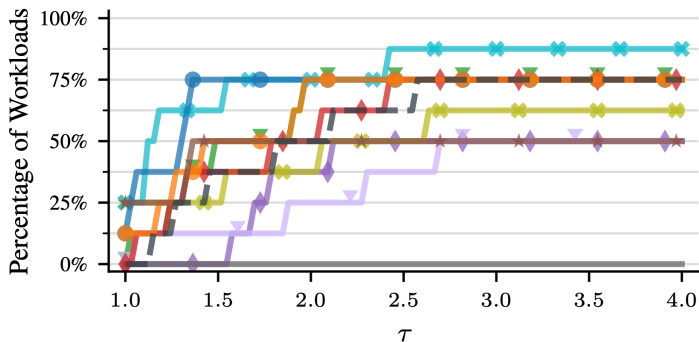
Сводка по фиксированным базовым задачам в бенчмарке AlgoPerf. В качестве функций потерь используются кросс-энтропия (CE), средняя абсолютная ошибка (L1) и потери CTC (Connectionist Temporal Classification). Дополнительные метрики качества: индекс структурного сходства (SSIM), частота ошибок (ER и WER), средняя точность (mAP) и BLEU. Бюджет времени соответствует режиму External Tuning; в режиме Self-Tuning обучение может идти в 3× дольше.

Задача	Датасет	Модель	Потери	Метрика	Цель	Бюджет
Прогнозирование CTR	CRITEO 1TB	DLRMSMALL	CE	CE	0.123735	7703
Реконструкция MPT	FASTMRI	U-NET	L1	SSIM	0.7344	8859
Классификация изображений	IMAGENET	ResNet-50	CE	ER	0.22569	63,008
		ViT	CE	ER	0.22691	77,520
Распознавание речи	LIBRISPEECH	Conformer	CTC	WER	0.085884	61,068
		DeepSpeech	CTC	WER	0.119936	55,506
Предсказание свойств молекул	OGBG	GNN	CE	mAP	0.28098	18,477
Машинный перевод	WMT	Transformer	CE	BLEU	30.8491	48,151

Бенчмарк AlgoPerf

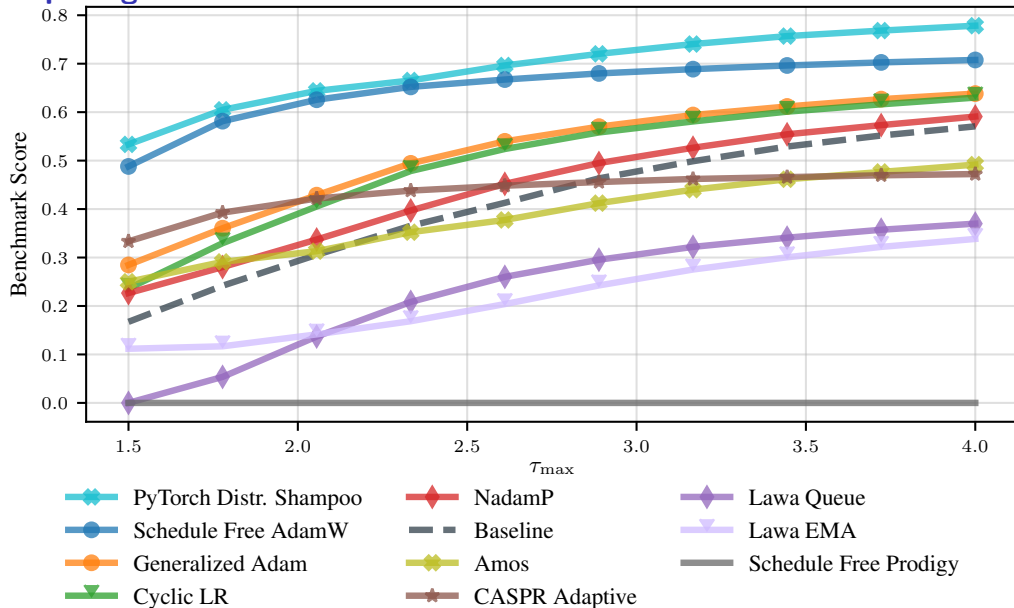
Submission	Line	Score
PYTORCH DISTRIBUTED SHAMPOO		0.7784
SCHEDULE FREE ADAMW		0.7077
GENERALIZED ADAM		0.6383
CYCLIC LR		0.6301
NADAMP		0.5909
BASELINE		0.5707
AMOS		0.4918
CASPR ADAPTIVE		0.4722
LAWA QUEUE		0.3699
LAWA EMA		0.3384
SCHEDULE FREE PRODIGY		0

(a) External tuning leaderboard



(b) External tuning performance profiles

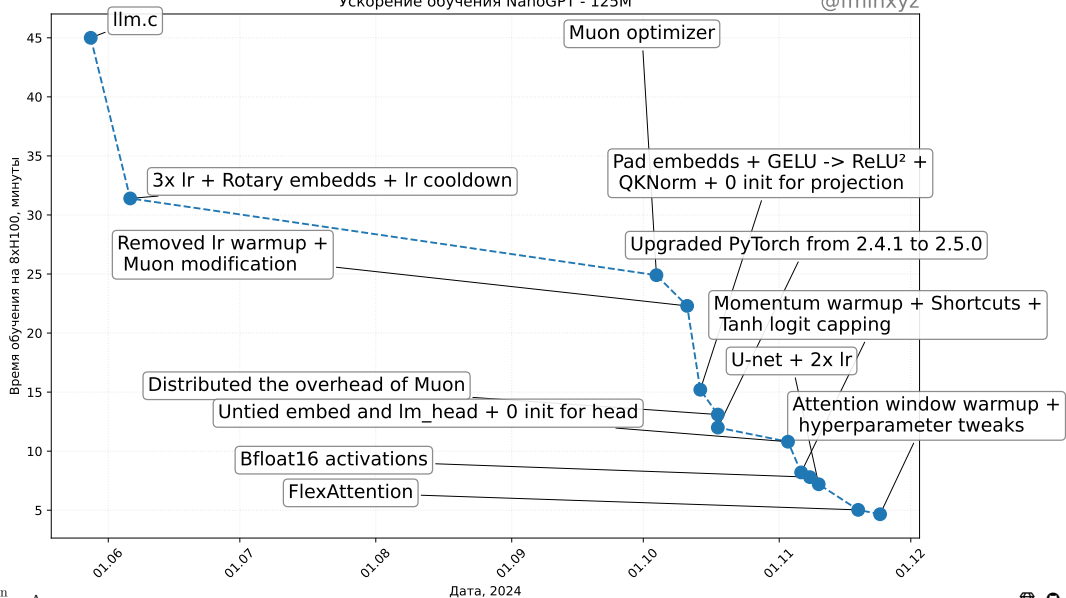
Бенчмарк AlgoPerf



Спидран NanoGPT

Ускорение обучения NanoGPT - 125M

@fminxyz



Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .

Замечания:

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистики $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.

Замечания:

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистики $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобуславливатели $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный корень матрицы)

Замечания:

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистики $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобуславливатели $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный корень матрицы)
4. Обновить параметры: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистики $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобуславливатели $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный корень матрицы)
4. Обновить параметры: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистики $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобуславливатели $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный корень матрицы)
4. Обновить параметры: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.
- Вычислительно дороже Adam, но по числу шагов может сходиться быстрее или приходить к лучшим решениям.

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистики $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобуславливатели $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный корень матрицы)
4. Обновить параметры: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.
- Вычислительно дороже Adam, но по числу шагов может сходиться быстрее или приходить к лучшим решениям.
- Требуется аккуратной и эффективной реализации (например, быстрого вычисления обратных корней матриц и работы с большими матрицами).

Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

Основная идея: аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровых произведений.

Для весовой матрицы $W \in \mathbb{R}^{m \times n}$ обновление использует предобуславливание через приближения статистических матриц $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистики $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобуславливатели $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный корень матрицы)
4. Обновить параметры: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.
- Вычислительно дороже Adam, но по числу шагов может сходиться быстрее или приходить к лучшим решениям.
- Требуется аккуратной и эффективной реализации (например, быстрого вычисления обратных корней матриц и работы с большими матрицами).
- Существуют варианты для тензоров разных форм, например для сверточных слоёв.

$$\begin{aligned}W_{t+1} &= W_t - \eta(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4} \\&= W_t - \eta(U S^2 U^\top)^{-1/4} (U S V^\top) (V S^2 V^\top)^{-1/4} \\&= W_t - \eta(U S^{-1/2} U^\top) (U S V^\top) (V S^{-1/2} V^\top) \\&= W_t - \eta U S^{-1/2} S S^{-1/2} V^\top \\&= W_t - \eta U V^\top\end{aligned}$$